

Aula 5 — Temporizadores

Quando o processador deixa de ser o relógio

Microcontroladores — DENE/UFMT

OBJETIVOS

- Justificar a existência de um temporizador a partir do erro medido de um relógio feito por espera bloqueante.
- Calcular pré-carga e divisor para um intervalo dado, a partir de T_{cy} , e dizer o papel de cada um.
- Comparar a contagem por instrução, por consulta ao indicador e por interrupção pelo erro que cada uma produz quando o processador tem outra tarefa.
- Reconhecer o temporizador como contador de eventos, cuja fonte pode ser o relógio interno ou um pino.

1 O relógio que atrasa

Encontro	Mecanismo	O que o processador estava fazendo
0	Laço vazio contado	Executando instruções que não fazem nada
2	Contagem de ciclos	O mesmo, agora com o número certo
3	<code>__delay_us</code> , espera cega	Nada, por 5200 ciclos
4	<code>while (ADCON0bits.GO)</code>	Nada, por 60 ciclos

Quatro mecanismos, uma única técnica: **gastar instruções até que o tempo passe**. Enquanto o processador só tem uma coisa a fazer, isso basta. O preço aparece na primeira vez que ele tem duas.

O programa abaixo é um relógio de segundos nos LEDs de PORTD. A linha `TAREFA()` representa **o resto do termostato** — ler o sensor, atualizar a tela, decidir —, aqui imitado por 300 ms de espera.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA() __delay_ms(300) /* "o resto do termostato" */

void main(void)
{
    uint8_t segundos = 0;

    LATD = 0xFF; /* LEDs apagados (ativos em 0) */
    TRISD = 0x00; /* LAT antes de TRIS: encontro 2 */

    for (;;) {
        __delay_ms(1000);
        segundos++;
        if (segundos > 59) {
            segundos = 0;
        }
        LATD = (uint8_t)~segundos;
        TAREFA();
    }
}
```

PREVISÃO

Cronometre 60 s com o relógio acima rodando. Quanto ele marca?

Escreva o número antes de rodar. Esta mesma pergunta vai ser feita a mais duas versões do programa ao longo da aula, e o placar fecha no fim.

Nenhum código desta aula traz `#pragma config`: os bits de configuração pertencem ao bootloader, e as diretivas seriam ignoradas em silêncio — **o código que veio antes do seu**.

NESTE KIT

O LED de RD0 é ativo em nível baixo (verificado na bancada), e daí o ~. O complemento supõe o mesmo para RD1 a RD7 — confirmar. Antes de escrever em LATD, as chaves que ligam os relés a PORTD precisam estar desligadas (item CH5-3/CH5-4, ainda em aberto).

O relógio atrasa porque o tempo é o código: cada volta dura 1000 ms de espera mais 300 ms de tarefa. Qualquer linha acrescentada ao laço muda o comprimento do segundo. Para que o segundo deixe de depender do programa, a contagem precisa sair do processador.

2 Um contador que anda sozinho

Um temporizador é um registrador que incrementa por conta própria, a cada pulso de uma fonte de contagem, sem executar instrução nenhuma. Quando passa do valor máximo, ele volta a zero e **levanta um indicador** — um bit num SFR.

CONCEITO

O processador não precisa acompanhar a contagem. Precisa apenas, de tempos em tempos, olhar o indicador.

O PIC18F4550 tem quatro: Timer0 e Timer1 (16 bits), Timer2 (8 bits, com comparador de período) e Timer3 (16 bits). Nesta aula interessa o Timer0. O Timer2, que sustenta o PWM, é o assunto do encontro 8.

2.1 A conta do pré-carregamento

A fonte de contagem interna é $F_{osc}/4$, ou seja, um incremento por ciclo de máquina: 250 ns. Um contador de 16 bits estoura em 65536 contagens, o que dá 16,4 ms.

Para um intervalo **menor**, não se espera o estouro desde zero: parte-se de um valor mais alto.

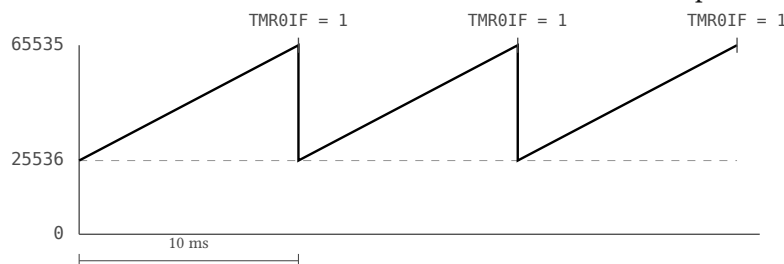
Para 10 ms, contando a 250 ns:

CONCEITO

$$N = 10 \text{ ms} / 250 \text{ ns} = 40000 \text{ contagens}$$

$$\text{pré-carga} = 65536 - 40000 = 25536 = 0x63C0$$

Escreve-se 25536 em TMR0, e o estouro acontece 40000 ciclos depois.



O processador não participa da subida. Ele só precisa aparecer nos instantes marcados — para zerar o indicador e recarregar 25536.

Figura 1: Um intervalo de 10 ms, três vezes. O tempo passa na diagonal; o processador só é convocado nas marcas.

2.2 E um segundo?

Um segundo são $1\text{ s} / 250\text{ ns} = 4000000$ contagens. Não cabe em dezesseis bits, e nenhuma pré-carga resolve: a pré-carga só **encurta** o caminho até o estouro.

Para alongá-lo, o Timer0 tem um **divisor** na entrada, que deixa passar um pulso a cada 2, 4, 8, até 256.

Com divisor 1:256, cada contagem vale $256 \cdot 250\text{ ns} = 64\text{ }\mu\text{s}$:

CONCEITO

$$N = 1\text{ s} / 64\text{ }\mu\text{s} = 15625\text{ contagens}$$

$$\text{pré-carga} = 65536 - 15625 = 49911 = 0xC2F7$$

Pré-carga e divisor são botões diferentes. O divisor escolhe o tamanho do passo; a pré-carga escolhe onde a contagem começa. O primeiro alonga, o segundo encurta.

A ordem das duas escritas na pré-carga não é estilo. TMR0H não é o byte alto do contador: é um **buffer**. Escrever nele não muda nada; a escrita em TMR0L é que transfere os dezesseis bits de uma vez.

ATENÇÃO

Escrever TMR0L primeiro carrega o byte baixo novo com o byte alto **anterior**, e o contador parte de um valor que ninguém pediu. O erro é intermitente, porque depende de qual era o valor anterior.

A leitura tem a simetria oposta: lê-se TMR0L primeiro, e é essa leitura que congela o byte alto no buffer.

3 O mesmo relógio, contado pelo hardware

A contagem vai para o Timer0. O laço principal passa a **consultar o indicador** TMR0IF a cada volta, e só age quando ele sobe.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA() __delay_ms(300)
#define PRECARGA 49911u /* 65536 - 15625: 1 s a 64 us */

static void t0_recarregar(void)
{
    TMR0H = (uint8_t)(PRECARGA >> 8); /* byte alto vai para o buffer */
    TMR0L = (uint8_t)(PRECARGA & 0xFF); /* esta escrita transfere os dois */
}

void main(void)
{
    uint8_t segundos = 0;

    LATD = 0xFF;
    TRISD = 0x00;

    t0_recarregar();
    T0CON = 0b10000111; /* ligado, 16 bits, relógio interno, divisor 1:256 */
    INTCONbits.TMR0IF = 0;

    for (;;) {
        if (INTCONbits.TMR0IF) { /* passou um segundo? */
            INTCONbits.TMR0IF = 0;
            t0_recarregar();
            segundos++;
        }
    }
}
```

```

        if (segundos > 59) {
            segundos = 0;
        }
        LATD = (uint8_t)~segundos;
    }
    TAREFA();
}
}

```

PREVISÃO

Mesma pergunta, mesma tarefa de 300 ms: em 60 s de cronômetro, quanto este relógio marca? Melhorou em relação ao primeiro? Ficou certo?

Melhora, mas não fica certo — e o erro tem uma forma curiosa.

O temporizador conta um segundo exato, levanta o indicador, e **continua contando**. Só que ninguém está olhando: o processador está no meio da tarefa. Ele só vê o indicador na volta seguinte do laço, e aí escreve a pré-carga **por cima** de tudo o que o contador andou nesse meio-tempo. O tempo de espera pelo olhar se perde, e se soma ao segundo.

Como a recarga acontece sempre no instante da consulta, o próximo estouro fica amarrado ao ritmo do laço. Com uma volta de 300 ms, as consultas caem em 300, 600, 900 e 1200 ms depois da recarga. O estouro acontece em 1000 ms, e quem o vê é a consulta de 1200 ms.

CONCEITO

Cada “segundo” dura exatamente **1,2 s**, e em 60 s o relógio marca 50. O erro não é aleatório: o segundo é arredondado para cima até um múltiplo da volta do laço.

São duas falhas diferentes, e vale separá-las.

NOTA

Deriva: o indicador é visto com atraso, e o atraso se soma a cada intervalo. É o que o relógio acima mostra.

Perda: se a volta do laço for **maior** que o intervalo, o indicador estoura duas vezes entre duas consultas. Como ele é um único bit, o segundo estouro some. O programa conta um evento onde houve dois.

As duas são silenciosas. Nada trava, nada reinicia — só o tempo fica errado.

3.1 A terceira forma, apresentada antes de definida

No encontro 0, main apareceu sem definição completa: uma função que nenhuma linha do seu programa chama. Quem a chama é o código que veio antes do seu, e aceitamos isso para poder começar.

A função abaixo é do mesmo tipo. Nenhuma linha do programa a chama: **quem a chama é o hardware**, no instante em que o indicador sobe, interrompendo o que quer que o processador esteja fazendo.

```

#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA()    __delay_ms(300)
#define PRECARGA    49911u

volatile uint8_t segundos = 0;

static void t0_recarregar(void)
{

```

```

    TMR0H = (uint8_t)(PRECARGA >> 8);
    TMR0L = (uint8_t)(PRECARGA & 0xFF);
}

/* Ninguém chama esta funcao. O hardware chama. */
void __interrupt() tratar(void)
{
    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        t0_recarregar();
        segundos++;
        if (segundos > 59) {
            segundos = 0;
        }
    }
}

void main(void)
{
    LATD = 0xFF;
    TRISD = 0x00;

    t0_recarregar();
    T0CON = 0b10000111;
    INTCONbits.TMR0IF = 0;
    INTCONbits.TMR0IE = 1;    /* o Timer0 pode interromper */
    INTCONbits.GIE = 1;      /* interrupcoes ligadas no chip */

    for (;;) {
        LATD = (uint8_t)~segundos;
        TAREFA();
    }
}

```

Três coisas aceitas sem definição, a pagar no encontro 7:

ATENÇÃO

- a palavra `__interrupt()`, que marca a função que o hardware chama;
- os bits `TMR0IE` e `GIE`, que decidem **quem** pode interromper;
- a palavra `volatile` em `segundos`, e a escolha de `uint8_t` em vez de `int`.

Não tente variar nenhuma das três nesta semana. O que interessa agora é o efeito, não o mecanismo.

Com a mesma tarefa de 300 ms, o relógio marca 60 em 60 s.

Repare no que **não** melhorou: os LEDs ainda são atualizados só a cada volta do laço, então o número aparece até 300 ms depois de mudar. Mas a **contagem** não atrasa mais, porque a recarga acontece microssegundos depois do estouro, e não depois da tarefa.

NOTA

Medir e mostrar viraram coisas separadas. Só a primeira precisa ser pontual.

O bootloader ocupa o início da memória de programa, e o vetor de interrupção da aplicação é deslocado junto com ela. O projeto do curso já está configurado para isso. Se a função de interrupção nunca executar, esse deslocamento é o primeiro suspeito — é o mesmo tipo de divergência entre “onde o hardware começa” e “onde o HEX começa” do encontro 2.

NESTE KIT

Nem a interrupção zera o erro. Entre o estouro e a escrita de `TMR0` ainda passam algumas dezenas de ciclos; a escrita inibe o incremento por mais dois ciclos (datasheet, seção do `Timer0`); e, com divisor, ela zera o que o divisor já tinha acumulado.

DIVERGÊNCIA

O resíduo é de microssegundos por segundo — da ordem da tolerância do próprio cristal. Existem formas de eliminá-lo por hardware, com um comparador que zera o temporizador sem intervenção do programa, e elas ficam fora deste curso.

3.2 O placar

Versão	Duração real de um “segundo”	Marca em 60 s
Laço com <code>__delay_ms</code>		
Consulta ao indicador		
Interrupção		

4 O contador não sabe o que é tempo

O Timer0 não mede tempo. Ele conta pulsos, e até agora os pulsos vinham do relógio interno. Mudando um bit de `T0CON`, eles passam a vir do pino `T0CKI`, que é `RA4`.

```
#include <xc.h>
#include <stdint.h>

void main(void)
{
    LATD = 0xFF;
    TRISD = 0x00;
    TRISAbits.TRISA4 = 1;    /* RA4 = T0CKI: entrada */

    TMR0L = 0;
    T0CON = 0b11101000;    /* ligado, 8 bits, fonte = pino T0CKI,
                             borda de subida, sem divisor */

    for (;;) {
        LATD = (uint8_t)~TMR0L;    /* o processador só mostra; quem conta é o hardware */
    }
}
```

O mesmo registrador, a mesma lógica de estouro, outra fonte. Com o relógio interno, contar pulsos é medir tempo; com um pino, é contar eventos — voltas de um eixo, pulsos de um medidor de energia, peças numa esteira.

CONCEITO

O processador não participa da contagem em nenhum dos dois casos.

PREVISÃO

Um botão está ligado a `RA4`, com resistor de pull-up: solto, o pino lê 1; apertado, lê 0. Você vai apertar o botão dez vezes.

- O contador incrementa ao apertar ou ao soltar?
- Depois de dez apertos, quanto os LEDs mostram?

Com `T0SE = 0`, a contagem acontece na borda de **subida**. Com o botão ligando o pino ao terra, a subida é o instante em que ele é **solto**.

ATENÇÃO

E dez apertos raramente dão dez. O contato mecânico repica: fecha e abre várias vezes em poucos milissegundos, e o contador — fiel — conta cada repique. O hardware não erra; o sinal é que não é o que parecia.

Esse ruído de contato é assunto do encontro 7. Por ora, basta registrar que o número medido depende do botão, e anotar quanto deu.

Verificar antes da aula: se algum botão do XM118 alcança `RA4`. Se não, um botão avulso na protoboard, entre `RA4` e o terra, com pull-up de 10 kΩ para 5 V.

NESTE KIT

5 Dois usos, e um terceiro

Uso	Fonte	O que o processador faz
Medir tempo	Relógio interno	Olha o indicador — ou é chamado quando ele sobe
Contar eventos	Pino T0CKI	Lê o resultado quando quiser

O terceiro uso inverte o sentido: em vez de receber pulsos, o contador **produz** uma forma de onda no pino, e o processador só diz a largura do pulso. É a modulação por largura de pulso, e ela volta no encontro 8, depois das interrupções.

6 Previsão para o R6

PREVISÃO

P1. O Timer0 vai marcar 10 ms, e um pino vai ser invertido a cada estouro. Que período você espera medir nesse pino?

P2. O laço que consulta o indicador vai ganhar uma tarefa de 7 ms. O período do pino muda? Para quanto?

P3. Um botão com pull-up (solto = 1) em RC0, e o Timer1 contando bordas de subida. O contador incrementa ao apertar ou ao soltar? Depois de dez apertos, quanto ele mostra?

P4. A ventoinha gira a cerca de 3000 rpm, e o tacômetro dá dois pulsos por volta. Quantos pulsos o Timer1 conta numa janela de 1 s?

P5. O mesmo programa conta os pulsos duas vezes: pelo Timer1 e por software, olhando RC0 a cada volta do laço. O laço vai ganhar uma tarefa de 8 ms. Qual das duas contagens muda, e para mais ou para menos?

Leve esta folha preenchida. Cada pergunta vale 0,4 — 2,0 no total do R6 —, avaliada pelo raciocínio, não por acertar o número.

SEM NOTA

7 Exercícios

Exercício 5.1. Calcule a pré-carga do Timer0 para um intervalo de 2 ms, com fonte interna e sem divisor.

TAREFA

(a) Quantas contagens?

(b) Qual valor escrever em TMR0?

(c) O mesmo intervalo é possível **sem** pré-carga, usando o divisor? Justifique.

Exercício 5.2. O relógio da aula usa divisor 1:256 para contar 1 s.

TAREFA

(a) Qual é o **menor** divisor que permite contar 1 s em dezesseis bits?

(b) Calcule a pré-carga para esse divisor e para 1:128.

(c) Os três divisores (o seu, 1:128 e 1:256) dão um segundo exato? Há motivo para preferir um deles?

Exercício 5.3. O relógio por consulta ao indicador marcou 50 em 60 s com uma tarefa de 300 ms.

TAREFA

(a) Escreva a duração real de um “segundo” em função da duração L de uma volta do laço.

(b) Quanto o relógio marca com $L = 250$ ms? E com $L = 260$ ms?

(c) Um colega testa com 250 ms, vê o relógio certo e conclui que o problema foi resolvido. O que você diz a ele?

Exercício 5.4. Um programa consulta TMR0IF dentro do laço principal, e o laço contém uma atualização de tela de 1,3 ms. O intervalo do temporizador é de 1 ms.

TAREFA

(a) O que acontece?

(b) O sintoma é visível ou silencioso?

(c) Proponha duas correções.

No encontro 6: avaliação integradora I, cobrindo os encontros 0 a 5. A interrupção aparece nela só como **efeito** — o placar do relógio —, nunca como mecanismo.

NOTA

No encontro 7: a dívida desta aula é paga. A função que o hardware chama, GIE e TMR0IE, e o porquê de `volatile` e de `uint8_t`. Junto vêm o repique que o contador externo revelou, e um problema que esta aula não tocou: tocar uma nota no buzzer e escrever na tela ao mesmo tempo.